Пермский институт (филиал) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г.В. Плеханова»

Кафедра информационных технологий и программирования

Практическая работа #2

Тема работы: Создание и тест простого калькулятора

Работу выполнила:
Карлагина Софья Владимировна

Группа: ИПС-11

Преподаватель: Берестов Дмитрий Борисович

Пермь 2026

Оглавление

Кафедра информационных технологий и программирования 1
Пермь 2026 1
Цель работы 3
Описание проекта 3
Структура проекта 3
Ход работы 4
1.1 Создание Библиотеки классов 4
1.2 Создание WPF 5
1.3 Логика работы калькулятора 8
1.4 Создание тестов 11
Заключение 12

Цель работы

Создать простой калькулятор с библиотекой классов на WPF и ещё с проверками чтобы посмотреть на работоспособность калькулятора.

Описание проекта

Был разработан проект Calculator – калькулятор на WPF с поддержкой базовых арифметических операций (сложение “+”, вычитание “-” и т.д.), возведения в степень, скобок и работы с десятичными числами. Интерфейс основан на панели компоновки Grid с кнопками для ввода цифр, операций и управления вычислениями. Для тестирования логики создан проект Test на MSTest.

Структура проекта

На рисунке 1 представлена структура проекта: библиотека классов, модульные тесты и приложение WPF.

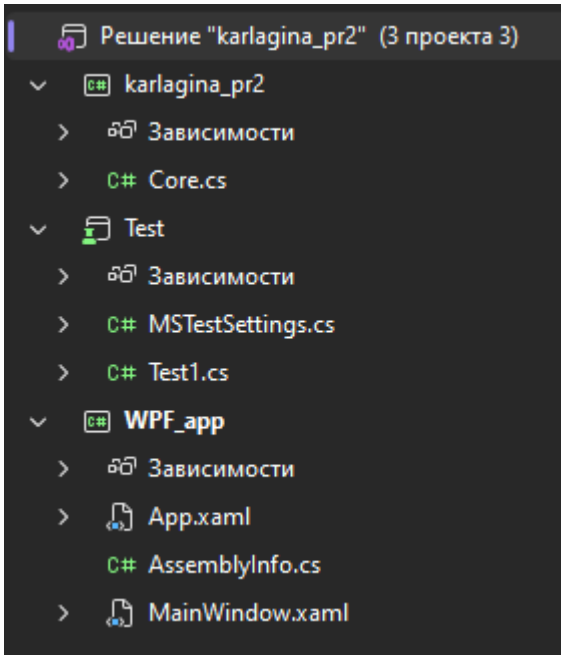

Рисунок 1 – Структура проекта karlagina_pr2

Ход работы

1.1 Создание Библиотеки классов

Первым делом создаём *Библиотеку классов (.NET) C#* и после создания пишем код для библиотеки (см. рисунок 2).

```
1   using System.Text;
2   using System.Threading.Tasks;
3
4   namespace karlagina_pr2
5   {
        Ссылок: 6
6       public class Core
7       {
            Ссылок: 6 | 5/5 passing
8           public static dynamic Exicute(dynamic first, char op, dynamic second)
9           {
10              dynamic result = 0;
11              switch (op)
12              {
13                  case '+':
14                      result = first + second;
15                      break;
16                  case '-':
17                      result = first - second;
18                      break;
19                  case '*':
20                      result = first * second;
21                      break;
22                  case '/':
23                      result = first / second;
24                      break;
25                  case '^':
26                      result = Math.Pow(first, second);
27                      break;
28                  default:
29                      result = 0;
30                      break;
31              }
32              return result;
33          }
34      }
35  }
36
```

Рисунок 2 – Основная логика в библиотеке классов

Код сделан в одном методе. Данный код после понадобится для построения следующих решений по выполнению практики и написанию дальнейших частей простого приложения.

Ниже будет представлен код из изображения, на котором будет основываться логика нашего простого приложения:

```
using System.Text;
using System.Threading.Tasks;

namespace karlagina_pr2
{
    public class Core
    {
        public static dynamic Exicute(dynamic first, char op, dynamic
second)
        {
            dynamic result = 0;
            switch (op)
            {
                case '+':
                    result = first + second;
                    break;
                case '-':
                    result = first - second;
                    break;
                case '*':
                    result = first * second;
                    break;
                case '/':
                    result = first / second;
                    break;
                case '^':
                    result = Math.Pow(first, second);
                    break;
                default:
                    result = 0;
                    break;
            }
            return result;
```

1.2 Создание WPF

Подключаем библиотеку классов к приложению WPF_app, добавляя ссылку на библиотеку через диспетчер ссылок. В проектах выбираем нужное нам решение. На рисунке 3 (см. ниже) оно будет отмечено галочкой.

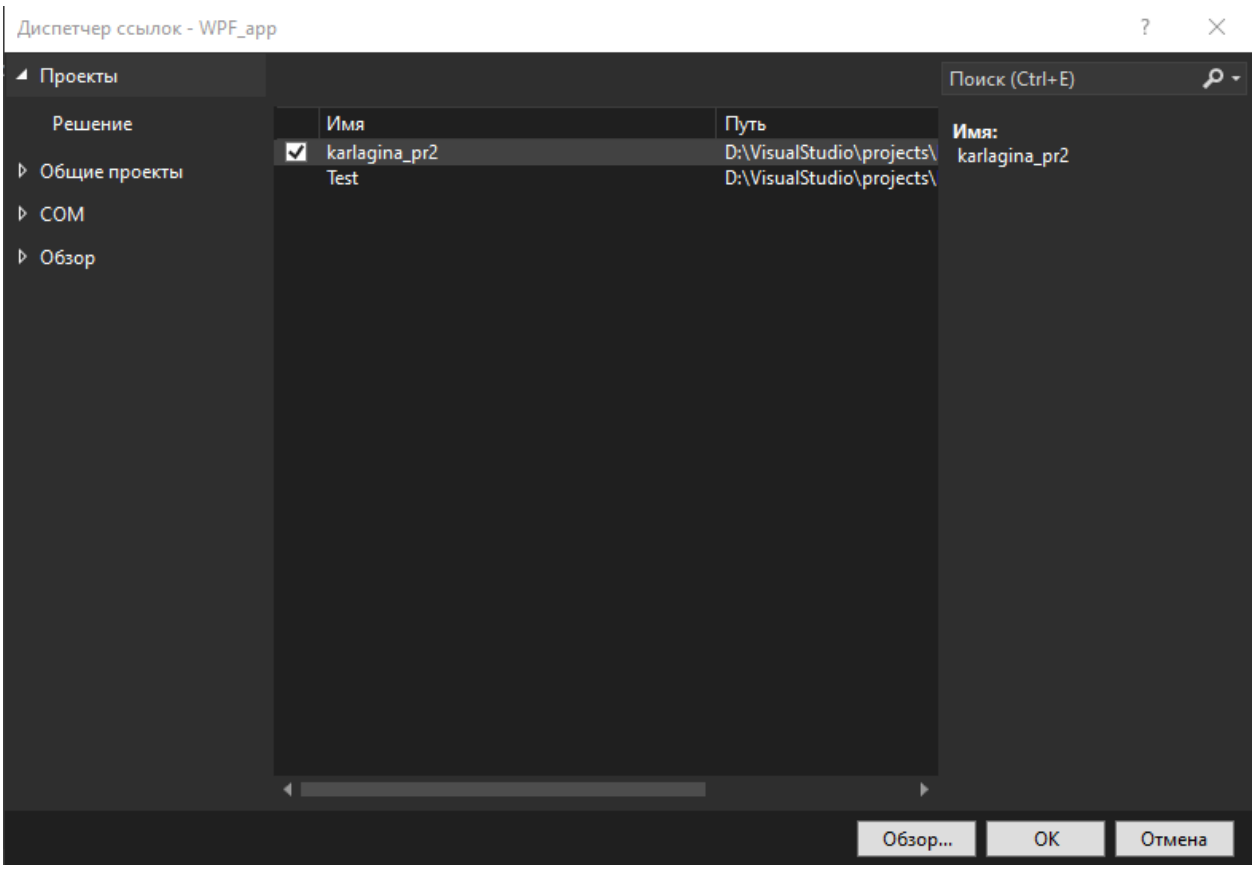

Рисунок 3 – Окно диспетчера ссылок WPF_app

Сначала создаём разметку, минимальные размеры окна приложения и даём ему более понятное название (см. рисунок 4).

```
    Title="Калькулятор" Height="600" Width="500">
<Grid x:Name="MainLayout">
    <Grid.RowDefinitions>
        <RowDefinition></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition></ColumnDefinition>
        <ColumnDefinition></ColumnDefinition>
        <ColumnDefinition></ColumnDefinition>
        <ColumnDefinition></ColumnDefinition>
    </Grid.ColumnDefinitions>
```

Рисунок 4 – Разметка окна приложения

С помощью элемента StackPanel объединяем верхнюю строку в единый контейнер и размещаем в нём два элемента TextBox. Первый TextBox

предназначен для ввода пользователем своего решения, второй — для отображения правильного ответа на поставленную задачу (см. рисунок 5):

```
<StackPanel Orientation="Vertical" Grid.Column="0" Grid.Row="0" Grid.ColumnSpan="4">
    <TextBox x:Name="tbCommandHistory" TextAlignment="Right" VerticalAlignment="Stretch" HorizontalAlignment="Stretch" FontSize="40" FontWeight="Normal"/>
    <TextBox x:Name="tbCommand" TextAlignment="Right" VerticalAlignment="Stretch" HorizontalAlignment="Stretch" FontSize="40" FontWeight="Normal"/>
</StackPanel>
```

Рисунок 5 – Создание текстовых полей

Создаём кнопки для калькулятора используя элемент кнопки Button (представляет шаблонный элемент управления "Кнопка") показанный на рисунке 6:

```
<Button x:Name="one" Grid.Column="0" Grid.Row="2" Content="1" FontSize="40"/>
<Button x:Name="two" Grid.Column="1" Grid.Row="2" Content="2" FontSize="40"/>
<Button x:Name="three" Grid.Column="2" Grid.Row="2" Content="3" FontSize="40"/>
<Button x:Name="four" Grid.Column="0" Grid.Row="3" Content="4" FontSize="40"/>
<Button x:Name="five" Grid.Column="1" Grid.Row="3" Content="5" FontSize="40"/>
<Button x:Name="six" Grid.Column="2" Grid.Row="3" Content="6" FontSize="40"/>
<Button x:Name="seven" Grid.Column="0" Grid.Row="4" Content="7" FontSize="40"/>
<Button x:Name="eight" Grid.Column="1" Grid.Row="4" Content="8" FontSize="40"/>
<Button x:Name="nine" Grid.Column="2" Grid.Row="4" Content="9" FontSize="40"/>
<Button x:Name="C" Grid.Column="0" Grid.Row="5" Content="C" FontSize="40"/>
<Button x:Name="zero" Grid.Column="1" Grid.Row="5" Content="0" FontSize="40"/>
<Button x:Name="dot" Grid.Column="2" Grid.Row="5" Content="." FontSize="40"/>
<Button x:Name="smooth" Grid.Column="3" Grid.Row="5" Content="=" FontSize="40"/>
<Button x:Name="plus" Grid.Column="3" Grid.Row="4" Content="+" FontSize="40"/>
<Button x:Name="minus" Grid.Column="3" Grid.Row="3" Content="-" FontSize="40"/>
<Button x:Name="multiply" Grid.Column="3" Grid.Row="2" Content="*" FontSize="40"/>
<Button x:Name="division" Grid.Column="3" Grid.Row="1" Content="/" FontSize="40"/>
<Button x:Name="RaistToPower" Grid.Column="2" Grid.Row="1" Content="^" FontSize="40"/>
<Button x:Name="RightParenthesis" Grid.Column="1" Grid.Row="1" Content=")" FontSize="40"/>
<Button x:Name="LeftParenthesis" Grid.Column="0" Grid.Row="1" Content="(" FontSize="40"/>
```

Рисунок 6 – Кнопки интерфейса приложения “Калькулятор”

Итоговый вид нашего калькулятора в конструкторе, воссозданный с помощью написанного выше кода, вы сможете увидеть на рисунке 7.

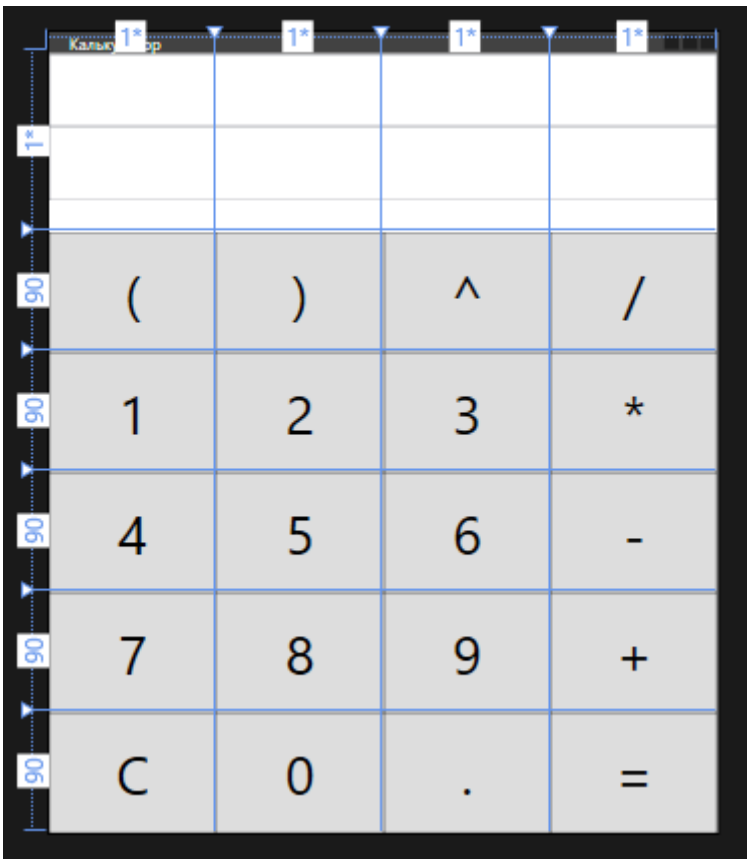

Рисунок 7 – Вид калькулятора в конструкторе

1.3 Логика работы калькулятора

Файл MainWindow.xaml.cs отвечает за взаимодействие пользователя с интерфейсом калькулятора. Весь код можно разделить на две логические части:

1. Код MainWindow на рисунке 8:

```
namespace WPF_app
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    Ссылок: 2
    public partial class MainWindow : Window
    {
        Ссылок: 0
        public MainWindow()
        {
            InitializeComponent();

            tbCommand.Clear();
            foreach (UIElement element in MainLayout.Children)
            {
                if (element is Button)
                {
                    ((Button)element).Click += Button_Click;
                }
            }
        }
        Ссылок: 1
```

Рисунок 8 – Код в MainWindow

InitializeComponent() — загружает визуальный интерфейс из файла

MainWindow.xaml (кнопки, сетку, текстовые поля). tbCommand.Clear() — очищает поле ввода при старте, чтобы калькулятор открывался с пустым экраном.

Автоматическая подписка на события:

1. Код перебирает все элементы внутри контейнера MainLayout.
2. Если элемент является кнопкой (Button), к ней автоматически «привязывается» метод Button_Click.
3. Результат: при нажатии на *любую* кнопку программа вызовет один и тот же метод обработки. Это позволяет добавлять новые кнопки в интерфейсе без переписывания кода.

2. Код в методе Button_Click

Метод обрабатывает нажатия всех кнопок и различные действия в зависимости содержимого кнопки.

- Получение текста из нажатой кнопки:

```
string TextButton = ((Button)e.OriginalSource).Content.ToString();
```

Определяется какая кнопка нажата и извлекается из неё текст.

- Обработка очищения:

```
if (TextButton == "C")
{
    tbCommandHistory.Clear();
    tbCommand.Clear();
}
```

- Получения выражения и поиск операторов (обработка “=”):

```
char[] ops = { '+', '-', '*', '/', '^' };

for (int i = 0; i < expr.Length; i++)
    if (Array.IndexOf(ops, expr[i]) >= 0)
```

Программа считывает выражение из поля ввода и последовательно ищет первый арифметический оператор (+, -, *, /, ^), запоминая его позицию и сам знак.

- Разделение на операнды и преобразование:

```
string num1Text = expr.Substring(0, pos);
string num2Text = expr.Substring(pos + 1);

double num1 = double.Parse(num1Text);
double num2 = double.Parse(num2Text);
```

Выражение разбивается на две части: первое число (от начала до оператора) и второе число (после оператора). Обе строки преобразуются в числа типа double.

- Вызов вычислений и вывод результата:

```
dynamic res = Core.Exicute(num1, op, num2);

tbCommandHistory.Text = $"{expr} =";
tbCommand.Text = res.ToString();
```

Числа и оператор передаются в метод Class1.Exicute, который выполняет математическую операцию. Результат отображается в основном поле, а само выражение с знаком «=» сохраняется в поле истории.

1.4 Создание тестов

В этот же проект создаём и добавляем проект с модульными тестами (см. рисунок 9):

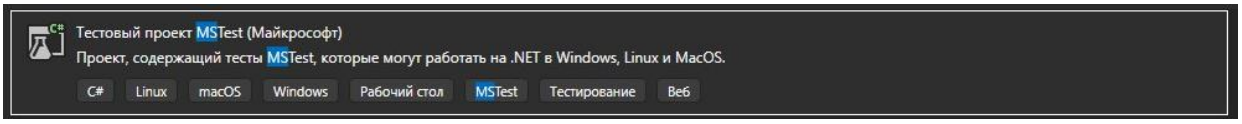

Рисунок 9 – Тестовый проект MSTest

Затем (с помощью ссылки на проект) подключаем тестовый проект к библиотеке классов и пишем наши тесты для проверки калькулятора:

```
using karlagina_pr2;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Mono.Cecil.Cil;
using System;

namespace Test
{
    [TestClass]
    public class CalculatorTests
    {
        [TestMethod]
        public void Test_Add()
        {
            dynamic result = Core.Exicute(8, '+', 3);
            Assert.AreEqual(11, result);
        }

        [TestMethod]
        public void Test_Subtract()
        {
            dynamic result = Core.Exicute(10, '-', 2);
            Assert.AreEqual(8, result);
        }

        [TestMethod]
        public void Test_Multiply()
        {
            dynamic result = Core.Exicute(7, '*', 9);
```

```
        Assert.AreEqual(63, result);
    }

    [TestMethod]
    public void Test_Divide()
    {
        dynamic result = Core.Exicute(25, '/', 5);
        Assert.AreEqual(5, result);
    }

    [TestMethod]
    public void Test_Power()
    {
        dynamic result = Core.Exicute(2, '^', 2);
```

На рисунке 10 мы можем увидеть, что наши тесты успешно пройдены:

Тестирование	Длительн...	Признаки	Сообщение об ошибке
Test (5)	245 мс		
Test (5)	245 мс		
CalculatorTests (5)	245 мс		
Test_Add	49 мс		
Test_Divide	49 мс		
Test_Multiply	49 мс		
Test_Power	49 мс		
Test_Subtract	49 мс		

Рисунок 10 – Успешное прохождение тестов

Заключение

В ходе работы мы создали простой калькулятор на платформе WPF поддерживает основные арифметические операции. Ключевое преимущество архитектуры — модульность: чёткое разделение пользовательского интерфейса и логики вычислений упрощает тестирование и последующую доработку кода. Для верификации корректности функционала создан тестовый проект на MSTest. Единая обработка всех кнопок одним методом позволила добиться компактности и гибкости программного кода.